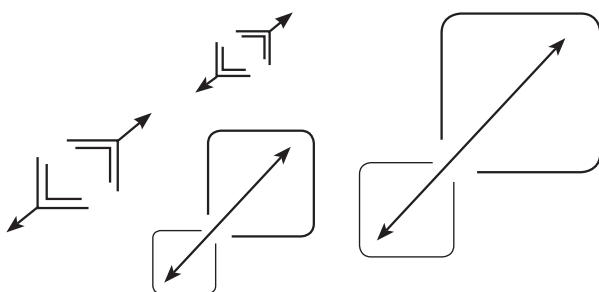


Autoscaling an Application with Kubernetes:

A Hands-On Guide

The resources used in applications sometimes fall short during times of peak demand. The HorizontalPodAutoscaler (HPA) in Kubernetes responds to such demands by deploying more pods.



In an orchestration scenario, and especially in Kubernetes, we often come across a situation where the resources assigned to a pod get utilised fully due to the peak demand for an application. Online shopping during Christmas or Diwali, or many people accessing music applications while going on a holiday over weekends, are examples of such situations.

A 'pod' is the smallest and simplest unit in the cluster that can be created, deployed, and managed. It represents a single instance of a running process within a cluster, and is used as the basic building block for deploying containerised applications.

A pod is a logical host for one or more containers. It is used to encapsulate one or more containers that are deployed together on the same host and share the same network name space. This means they can communicate with each other using local host. Containers within a pod can also share the same storage volumes, which allows them to access and modify the same files.

In such situations, it's highly important to manage autoscaling of our application. There are two types of scaling in Kubernetes.

- 1 **Horizontal scaling:** HPA (HorizontalPodAutoscaler)

basically works by adding more pods (Figure 1).

- 2 **Vertical scaling:** VPA (Vertical Pod Autoscaler) works by changing the pod size, as shown in Figure 2.

In this article we will discuss how to achieve horizontal scaling (using HPA) in an application. The prerequisites are:

- 1 Working k8s cluster
- 2 An already deployed and running metrics server
- 3 Running application. In this case the application is running using deployment.

HPA is a Kubernetes autoscaler, which scales up the replicas of a pod whenever a pod exceeds its

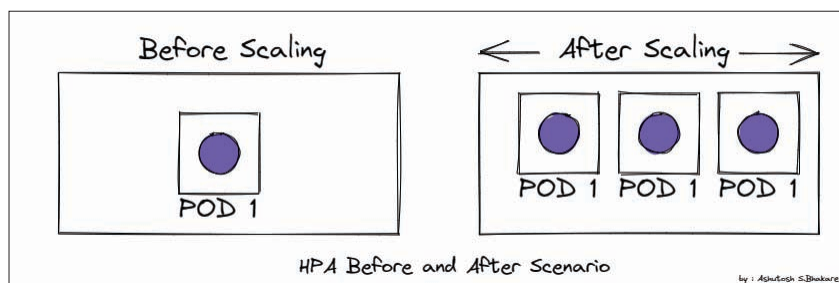


Figure 1: Horizontal scaling

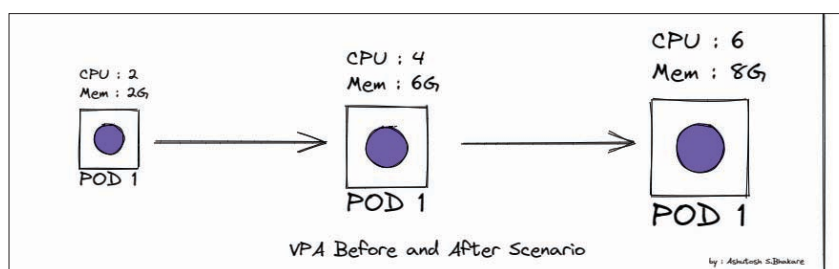


Figure 2: Vertical scaling